

NLP Tokenization Techniques



NLP1 (1).ipynb

File

explain the code -

The uploaded Jupyter notebook demonstrates basic **NLP (Natural Language Processing)** tasks using **NLTK** (Natural Language Toolkit). Here's a breakdown of what each code cell does:

◆ 1. Importing Libraries & Downloading NLTK Resources

```
python

import nltk
from nltk.tokenize import WhitespaceTokenizer, WordPunctTokenizer, TreebankWordTokenizer, TweetTokenizer
from nltk.stem import PorterStemmer, SnowballStemmer
from nltk.stem import WordNetLemmatizer
nltk.download('punkt')
nltk.download('wordnet')
nltk.download('omw-1.4')
nltk.download('stopwords')
```

- Imports various tokenizers, stemmers, and lemmatizers from NLTK.
- Downloads essential NLTK datasets.

◆ 2. Tokenization Techniques

```
python

text = "Welcome to the I2IT-NLP Page. Good Morning! Who is your favourite cryptocurrency influencer?"

# Whitespace Tokenization
wt = WhitespaceTokenizer()
whitespace_tokens = wt.tokenize(text)
```

- **WhitespaceTokenizer** splits text based on spaces only.

```
python

# WordPunctTokenizer
ppt = WordPunctTokenizer()
punctuation_tokens = ppt.tokenize(text)
```

- **WordPunctTokenizer** splits text on punctuation and whitespace.

python

```
# Treebank Tokenizer
treebank_tokenizer = TreebankWordTokenizer()
treebank_tokens = treebank_tokenizer.tokenize(text)
```

- More linguistically aware tokenizer (handles contractions, punctuation well).

python

```
# TweetTokenizer
tweet_tokenizer = TweetTokenizer()
tweet_tokens = tweet_tokenizer.tokenize(text)
```

- Optimized for social media (hashtags, emojis, etc.).

python

```
# MWETokenizer (Multi-Word Expressions)
mwe_tokenizer = MWETokenizer([('cryptocurrency', 'influencer')])
mwe_tokens = mwe_tokenizer.tokenize(whitespace_tokens)
```

- Combines specified word pairs (like "cryptocurrency influencer") into single tokens.

◆ 3. Stemming Techniques

python

```
# Porter Stemmer
porter = PorterStemmer()
porter_stemmed = [porter.stem(word) for word in whitespace_tokens]
```

- Reduces words to their root form, e.g., "influencer" → "influenc".

python

```
# Snowball Stemmer
snowball = SnowballStemmer(language='english')
snowball_stemmed = [snowball.stem(word) for word in whitespace_tokens]
```

- A more aggressive stemmer compared to Porter.

◆ 4. Lemmatization

python

```
lemmatizer = WordNetLemmatizer()
lemmatized_words = [lemmatizer.lemmatize(word) for word in whitespace_tokens]
```

- Converts words to their base dictionary form (e.g., "was" → "be"), but requires part-of-speech info for best results.

◆ 5. Stop Word Removal

python

```
from nltk.corpus import stopwords
stop_words = set(stopwords.words('english'))
filtered_lemmatized = [word for word in lemmatized_words if word.lower() not in stop_words]
```

- Removes common English words like “the”, “is”, “to” from the text.

Would you like me to run this notebook and show the outputs for each step?



write viva questions with answers of it

Here are some **viva questions and answers** based on the code in your NLP notebook:

✓ 1. What is tokenization in NLP?

Answer:

Tokenization is the process of splitting text into smaller units called tokens. Tokens can be words, phrases, or symbols. It is the first step in most NLP tasks.

✓ 2. What are different tokenizers used in NLTK?

Answer:

In this notebook, the following tokenizers are used:

- **WhitespaceTokenizer:** Splits text on spaces.
- **WordPunctTokenizer:** Splits text into words and punctuation separately.
- **TreebankWordTokenizer:** Uses rules based on the Penn Treebank corpus; handles contractions and punctuation better.
- **TweetTokenizer:** Specially designed for processing social media text.
- **MWETokenizer:** Combines multi-word expressions (e.g., "New York") into single tokens.

✓ 3. What is the difference between stemming and lemmatization?

Answer:

- **Stemming** cuts off prefixes/suffixes to get the root form of a word (e.g., "playing" → "play").
- **Lemmatization** converts a word to its dictionary base form (e.g., "better" → "good"). It is more accurate and meaningful but requires more processing.

✓ 4. What are Porter and Snowball stemmers?

Answer:

- **Porter Stemmer:** An older stemming algorithm with simple rules.
 - **Snowball Stemmer:** A newer and more accurate version developed by the same author. It supports multiple languages and produces better results.
-

✓ 5. What are stop words, and why are they removed?

Answer:

Stop words are common words like "is", "the", "and", which usually don't carry significant meaning in NLP tasks. Removing them helps reduce noise and improves model performance.

✓ 6. What is the purpose of `MWETokenizer``?

Answer:

`MWETokenizer`` is used to combine specific sequences of words into a single token, such as turning "cryptocurrency influencer" into one token. This helps preserve the meaning of phrases.

✓ 7. Why do we use `nlTK.download()`` in the beginning?

Answer:

It ensures that the necessary datasets and resources (like `punkt``, `wordnet``, `stopwords``) are available locally for tokenization, lemmatization, and stop word removal.

✓ 8. What would happen if we don't remove stop words in a task like sentiment analysis?

Answer:

If stop words are not removed, they may add unnecessary noise to the data, reduce model accuracy, and increase computational cost without improving understanding.

✓ 9. What is the output of `TweetTokenizer``? How is it different?

Answer:

`TweetTokenizer`` handles emojis, hashtags, usernames, and contractions more effectively, making it suitable for social media text. Its output is more robust for informal content.

✓ 10. What is the role of `WordNetLemmatizer``?

Answer:

`WordNetLemmatizer`` uses the WordNet lexical database to return the base (lemma) of a word, which can help normalize words for better analysis.

Would you like these questions formatted as a printable document or with diagrams for explanation?



yes

Your viva questions and answers have been compiled into a printable Word document.



Would you like a version with diagrams or visual examples included as well?



give more advance questions

Here are some **advanced viva questions with answers** related to the concepts used in your NLP notebook, ideal for impressing during technical evaluations or project presentations:

◆ 1. How does ``TreebankWordTokenizer`` handle contractions and punctuation?

Answer:

``TreebankWordTokenizer`` follows the Penn Treebank conventions. It separates punctuation from words and splits contractions into their components. For example, "don't" is split into "do" and "n't".

◆ 2. Explain the concept and use-case of ``MWETokenizer``.

Answer:

``MWETokenizer`` is used for combining Multi-Word Expressions (MWEs), such as "New York" or "machine learning", into a single token. This preserves semantic meaning during processing. It's useful in information retrieval, named entity recognition, and text classification.

◆ 3. What is the difference between ``WordPunctTokenizer`` and ``TreebankWordTokenizer``?

Answer:

- ``WordPunctTokenizer`` splits tokens based on punctuation (e.g., "don't" becomes "don" and "'t").
- ``TreebankWordTokenizer`` uses smarter heuristics to split contractions and handle punctuation more accurately.

◆ 4. How does lemmatization benefit downstream NLP tasks compared to stemming?

Answer:

Lemmatization ensures that words are reduced to valid dictionary forms, which improves semantic understanding and performance in tasks like text classification, sentiment analysis, and search relevance, while stemming may distort word meaning.

◆ 5. Why might `TweetTokenizer` be essential for real-world applications?

Answer:

TweetTokenizer is optimized for noisy, informal text. It handles:

- Emojis 😊
- Hashtags #AI
- User mentions @user
- Contractions and repeated characters

This makes it suitable for analyzing tweets, chats, and social media posts.

◆ 6. What are the limitations of using only rule-based tokenizers?**Answer:**

Rule-based tokenizers like those in NLTK:

- Do not understand language context.
- May split incorrectly for ambiguous words.
- Cannot handle multilingual or noisy text effectively.

Machine learning or deep learning-based tokenizers (like in spaCy or BERT) are better for complex language tasks.

◆ 7. Can lemmatization be done without POS tagging? What are the trade-offs?**Answer:**

Yes, but it's less accurate. For example, "saw" could be a noun or verb. Without POS tags, the lemmatizer may return the wrong base form. Including POS tags improves accuracy.

◆ 8. How does stop word removal affect tasks like Named Entity Recognition (NER)?**Answer:**

Removing stop words may negatively impact NER if entities include those words (e.g., "Bank of America"). Therefore, stop word removal should be applied selectively depending on the task.

◆ 9. What are some modern alternatives to NLTK for NLP tasks?**Answer:**

- **spaCy** – Faster, industrial-strength NLP library with deep learning models.
 - **Transformers (Hugging Face)** – Pretrained models like BERT, RoBERTa for advanced language understanding.
 - **Stanza** – NLP toolkit from Stanford for multilingual processing.
-

◆ 10. How can tokenization affect the performance of neural language models?

Answer:

Tokenization determines how input is represented. Poor tokenization:

- Increases vocabulary size.
- Reduces generalization.
- Affects embeddings quality.
Subword tokenization (e.g., WordPiece or BPE) used in models like BERT helps balance vocabulary size and expressiveness.

Would you like these advanced Q&A added to the same Word document as before?